DS Lab Assignment - 2: Odd 2026

Vaibhav Sharma (2501030279)

August 1, 2026

All source files here: Github Repository

Answer 1

```cpp
#include <bits/stdc++.h>
using namespace std;
class Node {
    public:
    int data;
    Node* next;
    Node(int value) {
        data = value;
        next = nullptr;
    }
};
class LinkedList {
    private:
    Node* head;
    public:
    LinkedList() {
        head = nullptr;
    }
    void insertHead(int value) {
        Node* newNode = new Node(value);
        if (head==NULL){
            head=newNode;
        }
        else{
            Node* curr=head;
            while(curr->next!=NULL){
                curr=curr->next;
            }
            curr->next=newNode;
        }
    }
    void cnt() {
        Node * temp=head;
        float ans=0;
        int c=0;
        while(temp!=NULL){
            c++;
            ans+=temp->data;
            temp=temp->next;
        }
        cout<<"Total number of nodes are:"<<c<<" and the average
           is:"<<fixed<<setprecision(2)<<ans/c<<"\n";
    }
    void last(int x) {
        Node * temp=head;
        int c=0;
        while(temp!=NULL){
            c++;
            temp=temp->next;
```

```
49          }
50          temp=head;
51          for (int i=0;i<c-x;i++){
52              temp=temp->next;
53          }
54          while(temp!=NULL){
55              cout<<temp->data<<" ";
56              temp=temp->next;
57          }
58          cout<<"\n";
59      }
60      void middle() {
61          Node* fast=head;
62          Node* slow=head;
63          if(head==NULL){
64              return;
65          }
66          while(fast!=NULL && fast->next!=NULL){
67              fast=fast->next->next;
68              slow=slow->next;
69          }
70          if(slow->data%2==0) {
71              cout<<"The middle element "<<slow->data<<" is even"<<
                    "\n";
72          }
73          else {
74              cout<<"The middle element "<<slow->data<<" is odd"<<"
                    \n";
75          }
76      }
77      void insertN(int n) {
78          int value;
79          for (int i = 0; i < n; i++) {
80              cout << "Enter data " << i + 1 << ": ";
81              cin >> value;
82              insertHead(value);
83          }
84      }
85      void display(int m) {
86          Node* temp = head;
87          for (int i=0;i<m;i++){
88              cout<<temp->data<<" ";
89              temp=temp->next;
90          }
91          cout<<"\n";
92      }
93      void deleteNode(int info) {
94          if (head == nullptr) {
95              cout << "No such value exists\n";
96              return;
97          }
98          if (head->data == info) {
99              Node* temp = head;
100             head = head->next;
101             delete temp;
102             return;
103         }
104         Node* curr = head;
105         Node* temp = head->next;
106         while (temp != nullptr) {
107             if (temp->data == info) {
108                 curr->next = temp->next;
109                 delete temp;
110                 return;
111             }
112             curr = temp;
113             temp = temp->next;
114         }
115         cout << "No such value exists\n";
116     }
```

```
117     void interchangePairs(int a1, int a2, int b1, int b2) {
118         if (head == NULL || head->next == NULL)
119             return;
120         Node *prev1 = NULL, *first1 = NULL;
121         Node *prev2 = NULL, *first2 = NULL;
122         Node *prev = NULL;
123         Node *curr = head;
124         while (curr && curr->next) {
125             if (curr->data == a1 && curr->next->data == a2) {
126                 prev1 = prev;
127                 first1 = curr;
128                 break;
129             }
130             prev = curr;
131             curr = curr->next;
132         }
133         prev = NULL;
134         curr = head;
135         while (curr && curr->next) {
136             if (curr->data == b1 && curr->next->data == b2) {
137                 prev2 = prev;
138                 first2 = curr;
139                 break;
140             }
141             prev = curr;
142             curr = curr->next;
143         }
144         if (first1 == NULL || first2 == NULL)
145             return;
146         if (first1 == first2)
147             return;
148         Node *second1 = first1->next;
149         Node *after1 = second1->next;
150         Node *second2 = first2->next;
151         Node *after2 = second2->next;
152         if (after1 == first2) {
153             if (prev1)
154                 prev1->next = first2;
155             else
156                 head = first2;
157             second2->next = first1;
158             second1->next = after2;
159             first1->next = second2;
160         }
161         else if (after2 == first1) {
162             if (prev2)
163                 prev2->next = first1;
164             else
165                 head = first1;
166             second1->next = first2;
167             second2->next = after1;
168             first2->next = second1;
169         }
170         else {
171             if (prev1)
172                 prev1->next = first2;
173             else
174                 head = first2;
175             if (prev2)
176                 prev2->next = first1;
177             else
178                 head = first1;
179             second2->next = after1;
180             second1->next = after2;
181             swap(first1->next, first2->next);
182         }
183     }
184 };
185 int main() {
186     LinkedList list;
187     int n;
```

```
    cout << "Enter number of nodes: ";
    cin >> n;
    list.insertN(n);
    list.cnt();
    cout<<"Enter number of nodes to display: ";
    int m;
    cin>>m;
    list.display(m);
    list.middle();
    int x;
    cout<<"Enter the node number from last: ";
    cin>>x;
    list.last(x);
    int data;
    cout<<"Enter the data to delete";
    cin>>data;
    list.deleteNode(data);
    cout<<"The list after deletion:\n";
    list.display(4);
    list.interchangePairs(2,3,4,5);
    return 0;
}
```

Answer 2

```
#include <iostream>
#include <string>
using namespace std;
struct Node {
    string data;
    Node* next;
};
class LinkedList {
    private:
    Node* head;
    public:
    LinkedList() {
        head = NULL;
    }
    void insert(string value) {
        Node* newNode = new Node;
        newNode->data = value;
        newNode->next = NULL;
        if (head == NULL) {
            head = newNode;
            return;
        }
        Node* curr = head;
        while (curr->next != NULL)
            curr = curr->next;
        curr->next = newNode;
    }
    void printList() {
        if (head == NULL) {
            cout << "List is empty.\n";
            return;
        }
        Node* curr = head;
        while (curr != NULL) {
            cout << curr->data << " ";
            curr = curr->next;
        }
        cout << endl;
    }
    void printStartingWith(char ch) {
        Node* curr = head;
        bool found = false;
        while (curr != NULL) {
            if (!curr->data.empty() && curr->data[0] == ch) {
```

```
45                  cout << curr->data << endl;
46                  found = true;
47              }
48              curr = curr->next;
49          }
50          if (!found)
51              cout << "No string starts with '" << ch << "'." <<
                    endl;
52      }
53      void searchString(string key) {
54          Node* curr = head;
55          while (curr != NULL) {
56              if (curr->data == key) {
57                  cout << "\"" << key << "\" exists in the linked
                        list.\n";
58                  return;
59              }
60              curr = curr->next;
61          }
62          cout << "\"" << key << "\" does not exist in the linked
                list.\n";
63      }
64      void longestString() {
65          if (head == NULL) {
66              cout << "List is empty.\n";
67              return;
68          }
69          Node* curr = head;
70          string longest = curr->data;
71          while (curr != NULL) {
72              if (curr->data.length() > longest.length())
73                  longest = curr->data;
74              curr = curr->next;
75          }
76          cout << "Longest string: " << longest
77               << " (Length = " << longest.length() << ")" << endl;
78      }
79      void containsXYZ() {
80          Node* curr = head;
81          bool found = false;
82          while (curr != NULL) {
83              if (curr->data.find("xyz") != string::npos) {
84                  cout << "\"" << curr->data
85                       << "\" contains \"xyz\".\n";
86                  found = true;
87              }
88              curr = curr->next;
89          }
90          if (!found)
91              cout << "No string contains \"xyz\".\n";
92      }
93      void swapPositions(int p1, int p2) {
94          if (p1 == p2) {
95              cout << "Both positions are the same.\n";
96              return;
97          }
98          Node *node1 = NULL, *node2 = NULL;
99          Node* curr = head;
100         int pos = 1;
101         while (curr != NULL) {
102             if (pos == p1)
103                 node1 = curr;
104             if (pos == p2)
105                 node2 = curr;
106             curr = curr->next;
107             pos++;
108         }
109         if (node1 == NULL || node2 == NULL) {
110             cout << "Error: One or both positions do not exist.\n
                    ";
```

```
            return;
        }
        swap(node1->data, node2->data);
        cout << "Strings interchanged successfully.\n";
    }
};
int main() {
    LinkedList list;
    list.insert("apple");
    list.insert("banana");
    list.insert("xyzabc");
    list.insert("orange");
    list.insert("grapexyz");
    list.insert("mango");
    list.insert("apricot");
    cout << "Original Linked List:\n";
    list.printList();
    cout << "\n(a) Print all nodes:\n";
    list.printList();
    cout << "\n(b) Strings starting with 'a':\n";
    list.printStartingWith('a');
    cout << "\n(c) Search for \"orange\":\n";
    list.searchString("orange");
    cout << "\nSearch for \"kiwi\":\n";
    list.searchString("kiwi");
    cout << "\n(d) Longest string:\n";
    list.longestString();
    cout << "\n(e) Strings containing \"xyz\":\n";
    list.containsXYZ();
    cout << "\n(f) Swap positions 2 and 6:\n";
    list.swapPositions(2, 6);
    cout << "Updated Linked List:\n";
    list.printList();
    cout << "\nAttempting to swap positions 7 and 10:\n";
    list.swapPositions(7, 10);
    return 0;
}
```

Answer 3

```
#include <iostream>
#include <vector>
using namespace std;
struct Node {
    char data;
    Node *next;
};
class LinkedList {
    private:
    Node *head;
    public:
    LinkedList() {
        head = NULL;
    }
    void insert(char ch) {
        Node *newNode = new Node;
        newNode->data = ch;
        newNode->next = NULL;
        if (head == NULL) {
            head = newNode;
            return;
        }
        Node *temp = head;
        while (temp->next != NULL)
            temp = temp->next;
```

```
        temp->next = newNode;
    }
    void display() {
        Node *temp = head;
        while (temp != NULL) {
            cout << temp->data << " ";
            temp = temp->next;
        }
        cout << endl;
    }
    Node* getHead() {
        return head;
    }
    void removeSequence(char a, char b, char c) {
        Node *prev = NULL;
        Node *curr = head;
        while (curr && curr->next && curr->next->next) {
            if (curr->data == a &&
                curr->next->data == b &&
                curr->next->next->data == c) {
                Node *first = curr;
                Node *second = curr->next;
                Node *third = second->next;
                Node *after = third->next;
                if (prev == NULL)
                    head = after;
                else
                    prev->next = after;
                delete first;
                delete second;
                delete third;
                return;
            }
            prev = curr;
            curr = curr->next;
        }
    }
};
int main() {
    LinkedList list1, list2;
    cout << "Enter 10 characters for first linked list:\n";
    for (int i = 0; i < 10; i++) {
        char ch;
        cin >> ch;
        list1.insert(ch);
    }
    cout << "Enter 5 characters for second linked list:\n";
    for (int i = 0; i < 5; i++) {
        char ch;
        cin >> ch;
        list2.insert(ch);
    }
    cout << "\nFirst List : ";
    list1.display();
    cout << "Second List: ";
    list2.display();
    vector<char> v;
    Node *temp = list2.getHead();
    while (temp) {
        v.push_back(temp->data);
        temp = temp->next;
    }
    for (int i = 0; i <= 2; i++) {
        list1.removeSequence(v[i], v[i + 1], v[i + 2]);
    }
    cout << "\nFirst list after removal:\n";
    list1.display();
    return 0;
}
```

Answer 4

```
#include <bits/stdc++.h>
using namespace std;
class Node {
    public:
    int data;
    Node* next;
    Node(int value) {
        data = value;
        next = nullptr;
    }
};
class LinkedList {
    private:
    Node* rear;
    Node* head;
    public:
    LinkedList() {
        head = nullptr;
        rear = nullptr;
    }
    void insert(int value) {
        Node* newNode = new Node(value);
        if (head==NULL){
            rear=newNode;
            head=newNode;
            rear->next=head;
        }
        else{
            Node* curr=head;
            while(curr->next!=head){
                curr=curr->next;
            }
            curr->next=newNode;
            rear=newNode;
            rear->next=head;
        }
    }
    void display() {
        if (head==NULL){
            cout<<"Nothing to display\n";
        }
        if (head->next==head){
            cout<<head->data<<"\n";
            return;
        }
        cout<<head->data<<" ";
        Node * curr=head->next;
        while (curr!=head){
            cout<<curr->data<<" ";
            curr=curr->next;
        }
        cout<<"\n";
    }
    int cnt() {
        Node * temp=head;
        int c=1;
        while(temp!=rear){
            c++;
            temp=temp->next;
        }
        return c;
    }
    void hasNegative() {
        if (head == NULL) {
            cout << "List is empty.\n";
            return;
        }
        Node* curr = head;
```

```
69          do {
70              if (curr->data < 0) {
71                  cout << "Negative value found: " << curr->data <<
                        endl;
72                  return;
73              }
74              curr = curr->next;
75          } while (curr != head);
76          cout << "No negative values found.\n";
77      }
78      void countGreaterThan15() {
79          if (head == NULL) {
80              cout << "List is empty.\n";
81              return;
82          }
83          int count = 0;
84          Node* curr = head;
85          do {
86              if (curr->data > 15)
87                  count++;
88              curr = curr->next;
89          } while (curr != head);
90          cout << "Number of nodes with value greater than 15: " <<
                count << endl;
91      }
92      void deleteNode(int value) {
93          Node *curr = head;
94          Node *prev = rear;
95          while (curr->data != value) {
96              prev = curr;
97              curr = curr->next;
98          }
99          if (head == rear) {
100             delete head;
101             head = rear = NULL;
102         }
103         else if (curr == head) {
104             head = head->next;
105             rear->next = head;
106             delete curr;
107         }
108         else if (curr == rear) {
109             prev->next = head;
110             rear = prev;
111             delete curr;
112         }
113         else {
114             prev->next = curr->next;
115             delete curr;
116         }
117         cout << "Element deleted successfully.\n";
118     }
119     void updateValue(int oldValue, int newValue) {
120         Node* curr = head;
121         while (curr->data != oldValue)
122             curr = curr->next;
123         curr->data = newValue;
124         cout << "Element updated successfully.\n";
125     }
126     void insertn(int a,int b,int c){
127         if(a==b+1){
128             insert(c);
129             return;
130         }
131         if(a==1){
132             Node* newNode = new Node(c);
133             Node* temp=rear;
134             temp->next=newNode;
135             temp->next=head;
136             head=temp;
137             return;
```

```
        }
        Node* curr=head;
        for (int i=1;i<a-1;i++){
            curr=curr->next;
        }
        Node* newNode = new Node(c);
        Node* temp=curr->next;
        curr->next=newNode;
        curr=curr->next;
        curr->next=temp;
    }
};
int main(){
    LinkedList list;
    list.insert(1);
    list.insert(2);
    list.insert(3);
    list.insert(4);
    list.insert(5);
    list.display();
    cout<<"Numbers of nodes are:"<<list.cnt()<<"\n";
    list.insertn(4,list.cnt(),10);
    list.display();
    list.hasNegative();
    list.updateValue(10, 16);
    list.countGreaterThan15();
    list.deleteNode(3);
    return 0;
}
```

Answer 5

```
#include <bits/stdc++.h>
using namespace std;
class Node{
    public:
    int data;
    Node* prev;
    Node* next;
    Node(int value){
        data=value;
        next=NULL;
        prev=NULL;
    }
};
class LinkedList{
    private:
    Node* head;
    public:
    LinkedList(){
        head=NULL;
    }
    void insert(int value){
        Node* newNode=new Node(value);
        if(head==NULL){
            head=newNode;
            head->prev=NULL;
            head->next=NULL;
            return;
        }
        Node* curr=head;
        while(curr->next!=NULL){
            curr=curr->next;
        }
        curr->next=newNode;
        newNode->prev=curr;
    }
    void display(){
```

```
        if(head==NULL){
            return;
        }
        Node* curr=head;
        while(curr!=NULL){
            cout<<curr->data<<" ";
            curr=curr->next;
        }
        cout<<"\n";
    }
    void divisible(int m){
        if(head==NULL){
            return;
        }
        Node* curr=head;
        while(curr!=NULL){
            if((curr->data)%m==0){
                cout<<curr->data<<" is divisible by "<<m<<"\n";
            }
            curr=curr->next;
        }
    }
    void deleteGreater(int x){
        Node* curr = head;
        while(curr != NULL){
            if(curr->data > x){
                Node* temp = curr;
                if(curr == head){
                    head = curr->next;
                    if(head != NULL)
                        head->prev = NULL;
                }
                else{
                    curr->prev->next = curr->next;
                    if(curr->next != NULL)
                        curr->next->prev = curr->prev;
                }
                curr = curr->next;
                delete temp;
            }
            else{
                curr = curr->next;
            }
        }
    }
    int countBetweenDuplicates(){
        Node* first = head;
        while(first != NULL){
            Node* curr = first->next;
            int count = 0;
            while(curr != NULL){
                if(curr->data == first->data){
                    cout << "Number of elements between "
                         << first->data << " = "<< count << "\n";
                    return count;
                }
                count++;
                curr = curr->next;
            }
            first = first->next;
        }
        cout << "No duplicate values found\n";
        return 0;
    }
};
int main() {
    LinkedList list;
    list.insert(1);
    list.insert(2);
    list.insert(3);
```

```
    list.insert(4);
    list.insert(5);
    list.display();
    list.divisible(3);
    list.deleteGreater(3);
    list.display();
    list.countBetweenDuplicates();
    return 0;
}
```

Answer 6

```
#include <bits/stdc++.h>
using namespace std;
class Node{
    public:
    int data;
    Node* prev;
    Node* next;
    Node(int value){
        data=value;
        next=NULL;
        prev=NULL;
    }
};
class LinkedList{
    private:
    Node* head;
    public:
    LinkedList(){
        head=NULL;
    }
    void insert(int value){
        Node* newNode=new Node(value);
        if(head==NULL){
            head=newNode;
            return;
        }
        Node* curr=head;
        while(curr->next!=NULL){
            curr=curr->next;
        }
        curr->next=newNode;
        newNode->prev=curr;
    }
    void display(){
        Node* curr=head;
        while(curr!=NULL){
            cout<<curr->data<<" ";
            curr=curr->next;
        }
        cout<<"\n";
    }
    void ExtremeSwap(){
        if(head == NULL || head->next == NULL)
            return;
        Node* left = head;
        Node* right = head;
        while(right->next != NULL){
            right = right->next;
        }
        while(left != right && left->prev != right){
            swap(left->data, right->data);
            left = left->next;
            right = right->prev;
```

```
        }
    }
};
int main(){
    LinkedList list;
    list.insert(1);
    list.insert(2);
    list.insert(3);
    list.insert(4);
    list.insert(5);
    list.insert(6);
    list.insert(7);
    list.insert(8);
    cout<<"Original: ";
    list.display();
    list.ExtremeSwap();
    cout<<"After 1st call: ";
    list.display();
    list.ExtremeSwap();
    cout<<"After 2nd call: ";
    list.display();
    list.ExtremeSwap();
    cout<<"After 3rd call: ";
    list.display();
    list.ExtremeSwap();
    cout<<"After 4th call: ";
    list.display();
    return 0;
}
```

Answer 7

```
#include <bits/stdc++.h>
using namespace std;
class Node {
    public:
    int coefficient;
    int power;
    Node* next;
    Node(int c, int p) {
        coefficient = c;
        power = p;
        next = NULL;
    }
};
class Polynomial {
    private:
    Node* head;
    public:
    Polynomial() {
        head = NULL;
    }
    void insert(int coefficient, int power) {
        Node* newNode = new Node(coefficient, power);
        if(head == NULL) {
            head = newNode;
            return;
        }
        Node* curr = head;
        while(curr->next != NULL) {
            curr = curr->next;
        }
        curr->next = newNode;
    }
    void display() {
```

```
        Node* curr = head;
        while(curr != NULL) {
            cout << curr->coefficient << "x^" << curr->power;
            if(curr->next != NULL)
                cout << " + ";
            curr = curr->next;
        }
        cout << endl;
    }
    Polynomial add(Polynomial& p) {
        Polynomial result;
        Node* a = head;
        Node* b = p.head;
        while(a != NULL && b != NULL) {
            if(a->power == b->power) {
                result.insert(a->coefficient + b->coefficient, a
                    ->power);
                a = a->next;
                b = b->next;
            }
            else if(a->power > b->power) {
                result.insert(a->coefficient, a->power);
                a = a->next;
            }
            else {
                result.insert(b->coefficient, b->power);
                b = b->next;
            }
        }
        while(a != NULL) {
            result.insert(a->coefficient, a->power);
            a = a->next;
        }
        while(b != NULL) {
            result.insert(b->coefficient, b->power);
            b = b->next;
        }
        return result;
    }
};
int main() {
    Polynomial p1, p2, result;
    // 5x^3 + 4x^2 + 2
    p1.insert(5, 3);
    p1.insert(4, 2);
    p1.insert(2, 0);
    // 3x^3 + 2x^2 + 7x + 1
    p2.insert(3, 3);
    p2.insert(2, 2);
    p2.insert(7, 1);
    p2.insert(1, 0);
    cout << "Polynomial 1: ";
    p1.display();
    cout << "Polynomial 2: ";
    p2.display();
    result = p1.add(p2);
    cout << "Sum: ";
    result.display();
    return 0;
}
```

Answer 8

```cpp
#include <bits/stdc++.h>
using namespace std;
class Node {
    public:
    int coefficient;
    int power;
    Node* next;
    Node(int c, int p) {
        coefficient = c;
        power = p;
        next = NULL;
    }
};
class Polynomial {
    private:
    Node* head;
    public:
    Polynomial() {
        head = NULL;
    }
    void insert(int coefficient, int power) {
        Node* newNode = new Node(coefficient, power);
        if(head == NULL) {
            head = newNode;
            return;
        }
        Node* curr = head;
        while(curr != NULL) {
            if(curr->power == power) {
                curr->coefficient += coefficient;
                delete newNode;
                return;
            }
            curr = curr->next;
        }
        curr = head;
        while(curr->next != NULL) {
            curr = curr->next;
        }
        curr->next = newNode;
    }
    void display() {
        Node* curr = head;
        while(curr != NULL) {
            cout << curr->coefficient << "x^" << curr->power;
            if(curr->next != NULL)
                cout << " + ";
            curr = curr->next;
        }
        cout << endl;
    }
    Polynomial multiply(Polynomial& p) {
        Polynomial result;
        Node* a = head;
        while(a != NULL) {
            Node* b = p.head;
            while(b != NULL) {
                int coefficient = a->coefficient * b->coefficient
                    ;
                int power = a->power + b->power;
                result.insert(coefficient, power);
                b = b->next;
            }
            a = a->next;
        }
        return result;
    }
};
int main() {
    Polynomial p1, p2, result;
```

```
    // 2x^2 + 3x + 1
    p1.insert(2, 2);
    p1.insert(3, 1);
    p1.insert(1, 0);
    // 4x + 5
    p2.insert(4, 1);
    p2.insert(5, 0);
    cout << "Polynomial 1: ";
    p1.display();
    cout << "Polynomial 2: ";
    p2.display();
    result = p1.multiply(p2);
    cout << "Product: ";
    result.display();
    return 0;
}
```

Answer 9

```
#include <bits/stdc++.h>
using namespace std;
class Node {
    public:
    int data;
    Node* next;
    Node(int value) {
        data = value;
        next = NULL;
    }
};
class LinkedList {
    private:
    Node* head;
    public:
    LinkedList() {
        head = NULL;
    }
    void insert(int value) {
        Node* newNode = new Node(value);
        if(head == NULL) {
            head = newNode;
            return;
        }
        Node* curr = head;
        while(curr->next != NULL) {
            curr = curr->next;
        }
        curr->next = newNode;
    }
    void rotateLeft(int k) {
        if(head == NULL || head->next == NULL)
            return;
        int n = 1;
        Node* last = head;
        while(last->next != NULL) {
            last = last->next;
            n++;
        }
        k = k % n;
        if(k == 0)
            return;
        Node* curr = head;
        for(int i = 1; i < k; i++) {
            curr = curr->next;
        }
        Node* newHead = curr->next;
        last->next = head;
```

```
        curr->next = NULL;
        head = newHead;
    }
    void display() {
        Node* curr = head;
        while(curr != NULL) {
            cout << curr->data << " ";
            curr = curr->next;
        }
        cout << endl;
    }
};
int main() {
    LinkedList list;
    list.insert(10);
    list.insert(20);
    list.insert(30);
    list.insert(40);
    list.insert(50);
    list.insert(60);
    cout << "Original list: ";
    list.display();
    list.rotateLeft(2);
    cout << "After rotation: ";
    list.display();
    return 0;
}
```

Answer 10

```
#include <bits/stdc++.h>
using namespace std;
class Node{
    public:
    int data;
    Node* next;
    Node(int value){
        data = value;
        next = NULL;
    }
};
class LinkedList{
    private:
    Node* head;
    public:
    LinkedList(){
        head = NULL;
    }
    void insert(int value){
        Node* newNode = new Node(value);
        if(head == NULL){
            head = newNode;
            return;
        }
        Node* curr = head;
        while(curr->next != NULL){
            curr = curr->next;
        }
        curr->next = newNode;
    }
    void removeDuplicates(){
        if(head == NULL)
            return;
        Node* curr = head;
        while(curr->next != NULL){
            if(curr->data == curr->next->data){
                Node* temp = curr->next;
```

```
38                  curr->next = temp->next;
39                  delete temp;
40              }
41              else{
42                  curr = curr->next;
43              }
44          }
45      }
46      void display(){
47          Node* curr = head;
48          while(curr != NULL){
49              cout << curr->data << " ";
50              curr = curr->next;
51          }
52          cout << endl;
53      }
54  };
55  int main(){
56      LinkedList list;
57      list.insert(1);
58      list.insert(1);
59      list.insert(2);
60      list.insert(3);
61      list.insert(3);
62      list.insert(3);
63      list.insert(5);
64      list.insert(5);
65      list.insert(8);
66      cout << "Before: ";
67      list.display();
68      list.removeDuplicates();
69      cout << "After: ";
70      list.display();
71      return 0;
72  }
```